Anexo 18 — Monitor de progreso de entrenamiento

Archivo: Scripts/monitor_v14b.py

Para que sirvio:
Evalua periodicamente (cada 45 min) el ultimo checkpoint de un entrenamiento de cabeza en curso, reportando el angulo logrado para varios objetivos de prueba.

```
"""Monitor periodico de v14b_head_scratch: cada 45min toma el ultimo checkpoint
y reporta theta + HeadRot + HeadBase para 5 azimuths de test.

Corre en paralelo con el training (`train_ppo.py`).

Uso:
    python Scripts/monitor_v14b.py
        [--run-dir runs/ppo_dum_v14b_head_scratch]
        [--interval-sec 2700]

Output:
    Imprime a stdout y a `<run_dir>/progress_check.log`.
"""
import argparse
import glob
import os
import sys
import time
from datetime import datetime
from pathlib import Path

SCRIPTS_DIR = Path(__file__).resolve().parent
sys.path.insert(0, str(SCRIPTS_DIR))

import numpy as np
import mujoco
from stable_baselines3 import PPO

from rl_env import DUMHeadTrackingEnv


TARGETS_AZ_DEG = [0, 60, 90, 120, 150, -90, -150]


def find_latest_checkpoint(run_dir: str):
    """Devuelve el checkpoint con mas steps (o None si no hay ninguno)."""
    cps = glob.glob(os.path.join(run_dir, "checkpoint_*_steps.zip"))
    if not cps:
        return None
    return max(cps, key=lambda p: int(os.path.basename(p).split("_")[1]))


def quick_eval(model_path: str, log_fp=None) -> list:
    """Eval rapida: 5 targets, 100 steps cada uno. Devuelve list de (az, theta, rot, base)."""
    def log(msg):
        print(msg, flush=True)
        if log_fp:
            log_fp.write(msg + "\n")
            log_fp.flush()

    model = PPO.load(model_path)
    env = DUMHeadTrackingEnv(cone_az_deg=150, cone_el_deg=50)
    results = []
    for az_deg in TARGETS_AZ_DEG:
        obs, _ = env.reset(seed=42)
        az = float(np.deg2rad(az_deg))
        el = float(np.deg2rad(10))
        target_dir = env._rot_z(az) @ env._rot_x(el) @ env._forward_world_init
        target_dir /= np.linalg.norm(target_dir) + 1e-9
        target_world = env._head_pos_init + 0.6 * target_dir
        env._target_mode = "static"
        env._target_a = target_world
        env._target_world = target_world.copy()
        env.data.mocap_pos[env._target_mocap_idx] = target_world
        mujoco.mj_forward(env.model, env.data)
        # v14c: 400 steps (8s) — el actuator HeadRot tiene forcerange=2.11 y
        # damping=5.0 -> velocidad max ~0.42 rad/s. Para 160° rotation necesita ~6.6s.
        # Con 100 steps (2s) la medicion seria falsamente pesimista.
        for _ in range(400):
            obs = env._get_obs()
            action, _ = model.predict(obs, deterministic=True)
            obs, r, term, trunc, info = env.step(action)
        theta = float(np.rad2deg(env._angle_to_target()))
        rot = float(np.rad2deg(env.data.qpos[env._head_qpos_adr[2]]))
```

```
        base = float(np.rad2deg(env.data.qpos[env._head_qpos_adr[1]]))
        results.append((az_deg, theta, rot, base))
    return results


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--run-dir", type=str, default="runs/ppo_dum_v14b_head_scratch")
    p.add_argument("--interval-sec", type=int, default=45 * 60,
                   help="Segundos entre checks (default 45min=2700s)")
    p.add_argument("--max-checks", type=int, default=10,
                   help="Cantidad maxima de checks antes de salir")
    args = p.parse_args()

    log_path = os.path.join(args.run_dir, "progress_check.log")
    os.makedirs(args.run_dir, exist_ok=True)

    print(f"[monitor] iniciando. Run dir: {args.run_dir}")
    print(f"[monitor] intervalo: {args.interval_sec}s ({args.interval_sec/60:.0f}min)")
    print(f"[monitor] log a: {log_path}")
    print(f"[monitor] esperando primer checkpoint...")
    sys.stdout.flush()

    log_fp = open(log_path, "a", encoding="utf-8")
    log_fp.write(f"\n=== monitor iniciado {datetime.now().isoformat()} ===\n")
    log_fp.flush()

    for check_n in range(args.max_checks):
        time.sleep(args.interval_sec)
        cp = find_latest_checkpoint(args.run_dir)
        if cp is None:
            msg = f"[monitor {datetime.now():%H:%M:%S}] no hay checkpoint todavia"
            print(msg, flush=True)
            log_fp.write(msg + "\n")
            log_fp.flush()
            continue
        steps_str = os.path.basename(cp).split("_")[1]
        msg = f"\n[monitor {datetime.now():%H:%M:%S}] check #{check_n+1}  checkpoint={steps_str} steps"
        print(msg, flush=True)
        log_fp.write(msg + "\n")
        log_fp.write(f"{'az':>5s}  {'theta':>7s}  {'HeadRot':>9s}  {'HeadBase':>9s}  status\n")
        log_fp.flush()
        try:
            results = quick_eval(cp, log_fp=None)
        except Exception as e:
            err = f"[monitor] eval error: {e}"
            print(err, flush=True)
            log_fp.write(err + "\n")
            log_fp.flush()
            continue
        # Reportar
        ok_count = 0
        for az, theta, rot, base in results:
            ok = (theta < 15) and (abs(base) < 35)
            if ok:
                ok_count += 1
            line = f"{az:>+4d}°  {theta:>6.1f}°  {rot:>+8.1f}°  {base:>+8.1f}°  {'OK' if ok else 'FAIL'}"
            print("  " + line, flush=True)
            log_fp.write(line + "\n")
        summary = f"  SUMMARY: {ok_count}/{len(results)} OK\n"
        print(summary, flush=True)
        log_fp.write(summary)
        log_fp.flush()

    log_fp.close()
    print("[monitor] fin (max-checks alcanzado)")


if __name__ == "__main__":
    main()
```